



UNIVERSIDADE DO MINHO

MESTRADO EM ENGENHARIA INFORMÁTICA

Dados e Aprendizagem Automática

Grupo 27

Cláudia Rego Faria - PG60240

Gonçalo Gonçalves Barroso - PG58980

Maria Inês Matos - PG61533

Tomás Pinto Rodrigues - A104448

10 de janeiro de 2026

Conteúdo

1	Introdução	3
1.1	Problema	3
1.2	Objetivos	3
1.3	Metodologia	3
2	Exploração e Análise de Dados	5
3	Preparação e Transformação de Dados	7
3.1	Limpeza e Tratamento Inicial	7
3.2	Feature Engineering	7
3.3	Codificação de Variáveis (Encoding)	7
3.4	Alterações pós-Checkpoint	7
3.4.1	Atribuição de pesos às classes	8
3.4.2	Criação de novas features	8
4	Modelos Desenvolvidos	9
4.1	Random Forest (Baseline)	9
4.2	Random Forest + GridSearchCV	9
4.3	Random Forest + Gradient Boosting	9
4.4	Stacking	9
4.5	Bagging (StratifiedShuffleSplit)	10
4.6	Bagging + GridSearchCV	10
4.7	Boosting	10
4.8	Boosting + GridSearchCV	10
4.9	Support Vector Machine	10
4.10	XGBoost + RandomSearchCV	11
4.11	Rede Neural Artificial (NNA)	11
4.12	Otimização de Modelos de Boosting (GridSearchCV)	11
4.13	Ensemble Avançado (Voting Classifier 5 Modelos)	12
5	Análise e Comparação de Modelos	13
5.1	Divisão do Dataset para Validação	13
5.2	Resultados	13
5.3	Análise qualitativa das previsões	15
6	Conclusão	17

1 Introdução

1.1 Problema

O tráfego rodoviário é um fenómeno complexo, caracterizado por um comportamento dinâmico, não linear e fortemente dependente do contexto temporal e ambiental. Em meio urbano, fatores como a hora do dia, o dia da semana, as condições meteorológicas e a ocorrência de incidentes contribuem para variações significativas no fluxo de veículos, tornando a sua previsão um desafio exigente.

No âmbito deste trabalho, o problema em estudo consiste na previsão do estado do tráfego rodoviário na cidade do Porto, num determinado instante temporal. Para esse efeito, é utilizado um dataset real que cobre um período superior a um ano, contendo informação relativa ao tráfego e a condições meteorológicas. O estado do trânsito é representado através da variável *average_speed_diff*, que indica, de forma qualitativa, a diferença entre a velocidade máxima permitida e a velocidade efetivamente observada, permitindo classificar o tráfego em cinco níveis: *None*, *Low*, *Medium*, *High* e *Very_High*.

A correta previsão desta variável é relevante, pois permite obter uma visão global do estado do trânsito na cidade, podendo contribuir para uma melhor gestão da mobilidade urbana e para a mitigação de congestionamentos.

1.2 Objetivos

O objetivo principal deste trabalho é desenvolver e avaliar modelos de *Machine Learning* capazes de prever o estado do tráfego rodoviário na cidade do Porto.

De forma mais concreta, pretende-se analisar e explorar o dataset utilizado, identificando padrões, relações entre variáveis e possíveis limitações dos dados. Pretende-se ainda preparar e transformar os dados de modo a torná-los adequados para a modelação, bem como desenvolver diferentes modelos de classificação recorrendo a várias abordagens de Aprendizagem Automática. Adicionalmente, visa-se otimizar os modelos através de técnicas de ajuste de hiperparâmetros e avaliar o seu desempenho com base na métrica de *accuracy*. Por fim, pretende-se comparar criticamente os resultados obtidos em validação e identificar os modelos com melhor capacidade de generalização.

1.3 Metodologia

A metodologia seguida neste trabalho encontra-se organizada em várias etapas. Numa primeira fase, foi realizada uma análise exploratória dos da-

dos, com o objetivo de compreender a estrutura do dataset, as distribuições das variáveis e as relações existentes entre elas.

Seguidamente, procedeu-se à preparação e transformação dos dados, incluindo limpeza, tratamento de valores em falta, criação de novas *features* e codificação de variáveis categóricas. Esta etapa revelou-se fundamental para garantir a qualidade dos dados utilizados no treino dos modelos.

Posteriormente, foram desenvolvidos e testados diversos modelos de *Machine Learning*, recorrendo a técnicas como *Bagging*, *Boosting*, *Stacking* e *Ensembles*, bem como à otimização de hiperparâmetros através de *GridSearchCV* e *RandomSearchCV*. Os modelos foram avaliados com base na *accuracy* obtida em treino, validação e na submissão final na plataforma *Kaggle*.

Por fim, foi realizada uma análise comparativa dos resultados, permitindo identificar o modelo com melhor desempenho global e menor propensão ao *overfitting*, bem como discutir a sua aplicabilidade ao problema de previsão do tráfego rodoviário.

2 Exploração e Análise de Dados

Numa fase inicial foi realizada uma análise exploratória dos dados, com o objetivo de compreender melhor as relações existentes entre as variáveis numéricas do dataset fornecido.

Para esse efeito, recorremos a uma Matriz de Correlação (Figura 2.1) que permite analisar a intensidade e a direção da relação linear entre pares de variáveis, assumindo valores entre -1 e 1.

A análise da matriz mostra que as correlações são, em geral, fracas a moderadas, não existindo dependências lineares fortes entre a maioria das variáveis. Destaca-se uma correlação negativa moderada entre a temperatura média (*AVERAGE_TEMPERATURE*) e a humidade média (*AVERAGE_HUMIDITY*), um resultado coerente do ponto de vista meteorológico.

Esta matriz foi fundamental para confirmar que não existe multicolinearidade entre as variáveis numéricas, validando a sua utilização conjunta nos modelos de aprendizagem automática desenvolvidos posteriormente.

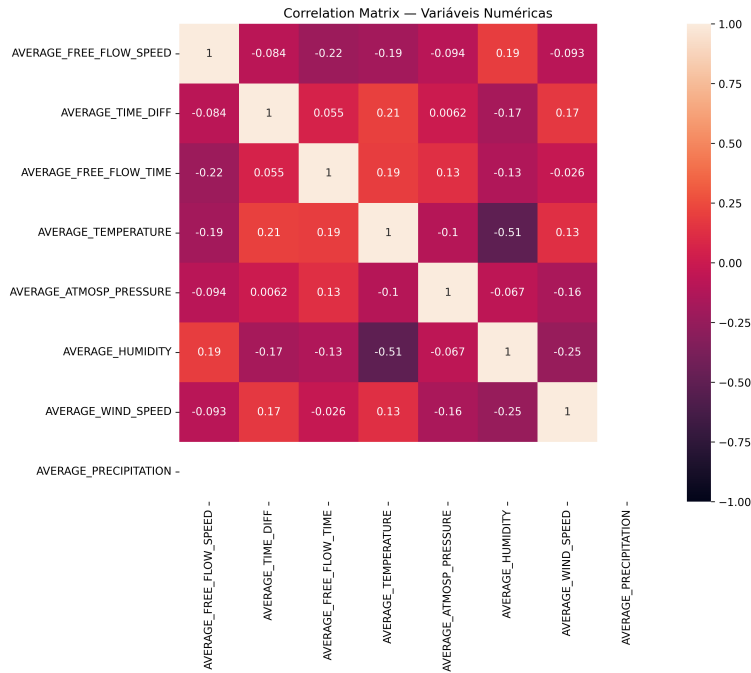


Figura 2.1: Matriz de Correlação das Variáveis Numéricas - Training Dataset

Complementarmente, recorreu-se a um *pairplot* (Figura 2.2) com o objetivo de analisar as distribuições univariadas e as interações entre pares de variáveis, segmentadas pela classe de velocidade (*AVERAGE_SPEED_DIFF*).

- **Distribuições (Diagonal):** As estimativas de densidade de kernel (KDE) indicam que a maioria das variáveis apresenta uma distribuição aproximadamente normal. A exceção é a variável *AVERAGE_TIME_DIFF*, que evidencia uma assimetria positiva acentuada.
- **Interações e Dispersão:** A significativa sobreposição entre as classes nos diagramas de dispersão sugere a inexistência de fronteiras lineares simples para a classificação do tráfego. Ainda assim, observa-se claramente uma correlação negativa entre a temperatura e a humidade.
- **Variância da Precipitação:** Verifica-se que a variável *AVERAGE_PRECIPITATION* apresenta uma variância praticamente nula na amostra analisada, o que sugere uma reduzida relevância estatística ou a necessidade de um pré-processamento específico antes da fase de modelação.

Esta análise visual reforça a ideia de que a complexidade inerente ao fenómeno do trânsito requer modelos capazes de capturar relações não lineares entre os fatores meteorológicos e o fluxo rodoviário.

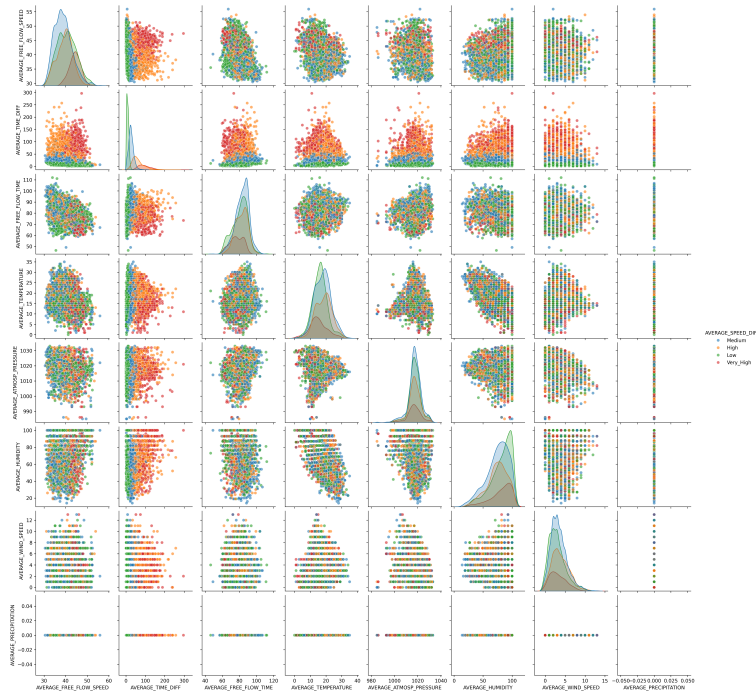


Figura 2.2: Pairplot das variáveis meteorológicas e de tráfego

3 Preparação e Transformação de Dados

3.1 Limpeza e Tratamento Inicial

Antes de qualquer modelação, foi efetuado um processamento base aos dados para garantir a sua qualidade e usabilidade pelos algoritmos.

Identificou-se que a variável *city_name* possuía um valor único em todas as observações. Por não apresentar variância nem acrescentar informação discriminativa para o modelo, esta coluna foi removida.

Relativamente aos dados em falta, detetou-se a existência de valores nulos na variável *average_precipitation*. Assumiu-se que a ausência de registo pluviométrico correspondia à não ocorrência de chuva, pelo que estes valores foram preenchidos com 0.

3.2 Feature Engineering

A variável temporal *record_date*, inicialmente em formato de texto, continha informação crucial para a previsão de tráfego, uma vez que este fenómeno segue padrões cíclicos diários e semanais.

Procedeu-se à conversão desta variável para o formato *datetime*, decompondo-a seguidamente em novas *features* numéricas individuais: mês, dia, hora e minuto. Foi ainda extraído o dia da semana (*weekday*), uma característica determinante para distinguir padrões de tráfego entre dias úteis e fins de semana. Após esta extração, a coluna original foi descartada.

3.3 Codificação de Variáveis (Encoding)

Uma vez que a maioria dos modelos de *Machine Learning* utilizados requer entradas numéricas, foi necessário converter as variáveis categóricas qualitativas.

Aplicou-se *Label Encoding* às variáveis *Luminosity*, *Sky_Situation* e *Rain_Condition*, mapeando as suas categorias textuais para valores inteiros. O mesmo processo foi aplicado à variável alvo *Speed_Diff*, transformando a escala ordinal ("None" a "Very_High") numa escala numérica de 0 a 4, permitindo assim o cálculo de métricas de erro e o treino dos classificadores.

3.4 Alterações pós-Checkpoint

Após o primeiro checkpoint foram introduzidas algumas alterações adicionais ao modelo de Boosting + GridSearch, com o objetivo de atribuir maior

relevância a determinadas variáveis e de criar *features* mais informativas a partir dos dados existentes.

3.4.1 Atribuição de pesos às classes

Devido ao desequilíbrio entre as classes da variável alvo, foram calculados pesos por classe utilizando a estratégia *balanced*, de forma a penalizar menos as classes minoritárias durante o treino. Estes pesos foram posteriormente convertidos em sample weights e utilizados no processo de treino e validação do modelo, de forma a garantir uma aprendizagem mais equilibrada.

3.4.2 Criação de novas features

Foram criadas novas variáveis a partir dos dados já existentes, com o objetivo de representar de forma mais explícita condições relevantes do contexto. Em particular, foram introduzidos indicadores binários associados à ocorrência de chuva e a períodos de hora de ponta, bem como combinações destas condições. Adicionalmente, foi aplicada uma seleção de features com base na sua importância estimada por um modelo de *AdaBoost*, permitindo reduzir o conjunto de variáveis utilizadas no treino.

Apesar destas alterações terem tornado o modelo conceptualmente mais informativo e robusto, não se observaram melhorias nos resultados finais, tendo 78.02% accuracy no treino e 77.35% de accuracy na validação, o que comparando com a versão anterior do modelo (presentes na Tabela 5.1) vê-se uma ligeira diminuição de accuracy.

4 Modelos Desenvolvidos

4.1 Random Forest (Baseline)

Como ponto de partida, foi treinado um modelo base de *Random Forest Classifier*. Este modelo foi instanciado com 100 árvores de decisão ($n_estimators=100$). Não foi aplicada qualquer otimização de hiperparâmetros nesta fase inicial, servindo este modelo apenas como referência de desempenho para as abordagens subsequentes. O treino final foi realizado sobre a totalidade do conjunto de dados antes da submissão.

4.2 Random Forest + GridSearchCV

Para melhorar o desempenho do modelo base, recorreu-se à técnica de *GridSearchCV* para a otimização de hiperparâmetros. Foi definida uma grelha de pesquisa que explorou diferentes combinações: número de estimadores (100, 200, 300), profundidade máxima das árvores (10, 20, 30 e sem limite), número mínimo de amostras para divisão (2, 5, 10) e nas folhas (1, 2, 4). O modelo final foi re-treinado com a melhor combinação de parâmetros identificada pela validação cruzada, maximizando a capacidade de generalização do modelo.

4.3 Random Forest + Gradient Boosting

Procurando beneficiar da combinação de diferentes algoritmos, foi desenvolvido um *Voting Classifier* que agrega um *Random Forest* e um *Gradient Boosting Classifier*. O *Gradient Boosting* foi configurado com uma *learning rate* de 0.05, 300 estimadores e *subsample* de 0.8 para reduzir o *overfitting*. O *Random Forest* complementar foi ajustado com 300 estimadores e profundidade máxima de 15. A combinação destes dois modelos permitiu capturar tanto as relações não-lineares complexas (via RF) como a correção iterativa de erros (via GB).

4.4 Stacking

Foram treinados modelos base de Regressão Logística (de modo a capturar relações lineares simples), Random Forest (de modo a capturar relações não lineares mais complexas) e XGBoost + GridSearchCV (focado em correção de erros), totalizando 1080 fits. O meta modelo final, que empilha as previsões dos modelos base, foi de Regressão Logística.

4.5 Bagging (StratifiedShuffleSplit)

Para este modelo, recorremos ao `BaggingClassifier` com uma `DecisionTree` como *estimator base*. O modelo foi treinado com um número fixo de *estimators* (100) e com *bootstrap* sobre os dados de treino. Os restantes parâmetros do modelo foram mantidos com valores padrão, não tendo sido aplicada qualquer técnica de optimização de hiperparâmetros.

4.6 Bagging + GridSearchCV

Numa fase posterior, foi aplicado o `GridSearchCV` para otimizar os hiperparâmetros do modelo Bagging. Foram testados diferentes valores para os *estimators* (100, 200 e 300), para *max_samples* (0.6, 0.8 e 1.0) e para *max_features* (0.6, 0.8 e 1.0). Foram ainda avaliadas as opções de utilização ou não de *bootstrap* ao nível das amostras e das variáveis. O modelo final foi treinado com a combinação de parâmetros através de *cross-validation*.

4.7 Boosting

Para este modelo, recorremos ao `AdaBoostClassifier` utilizando árvores de decisão rasas como *estimadores base*. O modelo foi treinado com 200 *estimators* e uma taxa de aprendizagem de 0.5, tendo os restantes parâmetros sido mantidos com valores padrão, não tendo sido aplicada qualquer técnica de optimização de hiperparâmetros.

4.8 Boosting + GridSearchCV

Posteriormente, recorreu-se ao *GridSearchCV* para otimizar os hiperparâmetros do *AdaBoostClassifier*. Foram testados diferentes valores para o número de *estimators* (50, 100 e 200) e para a taxa de aprendizagem (*learning_rate* igual a 0.1, 0.5 e 1.0). Adicionalmente, procedeu-se ao refinamento do *estimator base* através da definição da profundidade máxima (*max_depth* entre 1 e 3) e do parâmetro *min_samples_split*. O modelo final foi treinado com a melhor combinação de hiperparâmetros identificada através de validação cruzada (*cross-validation*).

4.9 Support Vector Machine

Como o SVM requer normalização de dados, as variáveis numéricas foram escaladas através do `StandardScaler`. Foram treinados 3 tipos de SVM: Linear (com *accuracy* de 71.97%), RBF (73.19%) e Polinomial (55.77%). Ao

modelo com melhor desempenho aplicou-se fine-tuning através do GridSearchCV, sendo o modelo final treinado com os melhores parâmetros encontrados.

4.10 XGBoost + RandomSearchCV

Para este modelo, foi usado o RandomSearchCV em vez do GridSearchCV, de modo a conservar tempo no seu treino. O modelo final foi treinado com os melhores parâmetros encontrados. Por ser o último modelo a ser testado e considerado aquele com maior potencial, foram testados diferentes ajustes aos seus parâmetros do RandomSearch: experimentou-se diferentes valores de *random state* (24, 34, 42, 123 e 2025) e ainda foram testadas divisões de 5000 e 7000 folds.

4.11 Rede Neural Artificial (NNA)

Para este modelo, implementou-se um MLPClassifier com uma arquitetura de duas camadas ocultas de 10 neurónios cada. Utilizou-se a função de ativação ReLU e o otimizador Adam, tendo o treino sido executado ao longo de 50 epochs (max_iter=50). Os dados foram previamente escalonados para assegurar a convergência do algoritmo, mantendo-se os restantes parâmetros com os valores padrão do sistema, não tendo sido aplicada qualquer técnica de optimização de hiperparâmetros.

4.12 Otimização de Modelos de Boosting (GridSearchCV)

Nesta etapa, focou-se na comparação e optimização de três algoritmos de *gradient boosting* de última geração: XGBoost, CatBoost e LightGBM. Implementou-se um ciclo de *GridSearchCV* (com *cv=3*) para cada um dos modelos individualmente.

- **XGBoost:** Testados *n_estimators* (100, 200), *max_depth* (5, 10, 15) e *learning_rate* (0.01, 0.1).
- **CatBoost:** Testados *iterations* (100, 200) e *depth* (4, 6, 8).
- **LightGBM:** Testados *num_leaves* (31, 50) juntamente com estimadores e profundidade.

Após a optimização individual, selecionaram-se os 3 modelos com melhor desempenho para compor um *Voting Classifier* com estratégia de *soft voting*.

4.13 Ensemble Avançado (Voting Classifier 5 Modelos)

Na abordagem mais robusta desenvolvida, construiu-se um *Ensemble* heterogêneo combinando cinco classificadores distintos: Random Forest, Gradient Boosting, XGBoost, LightGBM e CatBoost. Todos os modelos foram configurados com parâmetros ajustados para consistência (e.g., $n_estimators=300$ e $learning_rate=0.05$ para os modelos de boosting, exceto o CatBoost com 1000 iterações). O modelo final utilizou uma estratégia de *Soft Voting*, onde a previsão final é determinada pela média das probabilidades estimadas por cada um dos cinco modelos, permitindo uma decisão mais estável e resiliente a ruído nos dados.

5 Análise e Comparação de Modelos

5.1 Divisão do Dataset para Validação

Para cada modelo, os dados de treino foram divididos de forma a alocar 30% para a validação. Ou seja, uma parte serve para o treino real do modelo e a parte alocada à validação serve para a validação fixa. Este método permite identificar casos onde poderá estar a ocorrer *overfitting* antes da submissão do modelo.

5.2 Resultados

A seguinte tabela contém os valores finais de *accuracy* medidos para o treino, a validação e o valor obtido na submissão no *Kaggle* para cada modelo.

Modelo	Treino	Validação	Kaggle
Random Forest	100%	77.69%	80.222%
Random Forest + GridSearchCV	100%	78.57%	81.555%
XGBoost	100%	80.23%	82.888%
Bagging (StratifiedShuffleSplit)	99.98%	80.63%	79.777%
Bagging + GridSearchCV	100%	80.77%	79.777%
Boosting (DT)	69.99%	70.21%	67.555%
Boosting + GridSearchCV	80.14%	79.06%	82.000%
Stacking	100%	80.33%	80.666%
Gradient Boosting + Random Forest	79.74%	79.65%	81.555%
Support Vector Machine	–	76.86%	80.000%
XGBoost + RandomSearchCV	86.16%	81.03%	82.444%
Rede Neuronal	86.27%	75.28%	78.000%
Max Voting (CatBoost + XGBoost + LightGBM + RF)	93.44%	81.12%	81.333%
Max Voting (XGBoost + CatBoost + LightGBM + GridSearchCV)	97.42%	81.21%	81.555%

Tabela 5.1: Comparação da accuracy dos modelos

A análise do desempenho de cada modelo passa pela comparação dos valores de *accuracy* de treino e de validação. Quando estes valores estão distantes, particularmente quando o valor de treino é alto e o de validação significativamente mais baixo, estamos perante um caso bastante provável de *overfitting*. Nesta tabela constam os valores finais, pelo que ao longo

da realização do trabalho os modelos com maior potencial foram incrementalmente ajustados e afinados para reduzir ao máximo a probabilidade de *overfitting*. Fora os modelos iniciais, não foram detetadas instâncias de possível *underfitting*.

No final, o modelo que conjugava melhor desempenho em *accuracy* e menor probabilidade de *overfitting* foi o XGBoost + RandomSearchCV, com discrepância entre validações de apenas 5.13% e pontuação do *Kaggle* de 82.444%.

No final da competição do *Kaggle*, foram disponibilizadas as pontuações de *accuracy* para todos os modelos treinados. Segue-se a tabela contendo a comparação da pontuação pública e privada.

Modelo	Pontuação Pública	Pontuação Privada
Random Forest	80.222%	80.258%
Random Forest + Grid-SearchCV	81.555%	78.380%
XGBoost	82.888%	77.523%
Bagging (StratifiedShuffleSplit)	79.777%	78.000%
Bagging + GridSearchCV	79.777%	78.761%
Boosting (DT)	67.555%	66.571%
Boosting + GridSearchCV	82.000%	77.523%
Stacking	80.666%	78.761%
Gradient Boosting + Random Forest	81.555%	81.238%
Support Vector Machine	80.000%	73.714%
XGBoost + RandomSearchCV	82.444%	80.095%
Rede Neuronal	78.000%	73.333%
Max Voting (CatBoost + XGBoost + LightGBM + RF)	81.333%	79.333%
Max Voting (XGBoost + CatBoost + LightGBM + GridSearchCV)	81.555%	79.142%

Tabela 5.2: Comparação das pontuações privada e pública dos modelos

Observa-se que a maior parte dos modelos desceram entre 0 a 5% na pontuação privada. É possível notar que os modelos mais recente, e mais ajustados e refinados, foram os com menor variação na tabela. Alguns modelos chegaram ainda a subir ligeiramente de percentagem.

5.3 Análise qualitativa das previsões

No final, realizou-se uma pequena análise qualitativa das previsões. Considerou-se as previsões com maior *accuracy* no fim da competição, com 81.238%.

Na seguinte figura, pode ver-se a distribuição da *average_speed_diff* nas previsões.

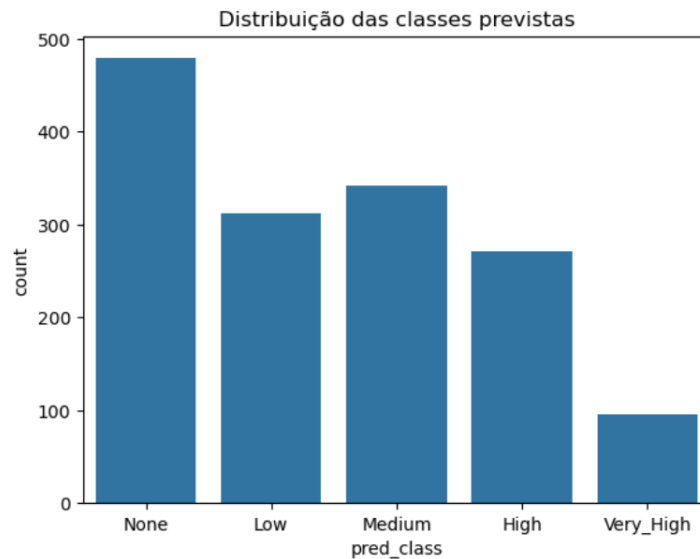


Figura 5.1: Distribuição da classe

Com base na seguinte figura, podemos observar que, se as previsões estivessem totalmente corretas, a maior correlação seria com *average_time_diff* e *average_wind_speed*.

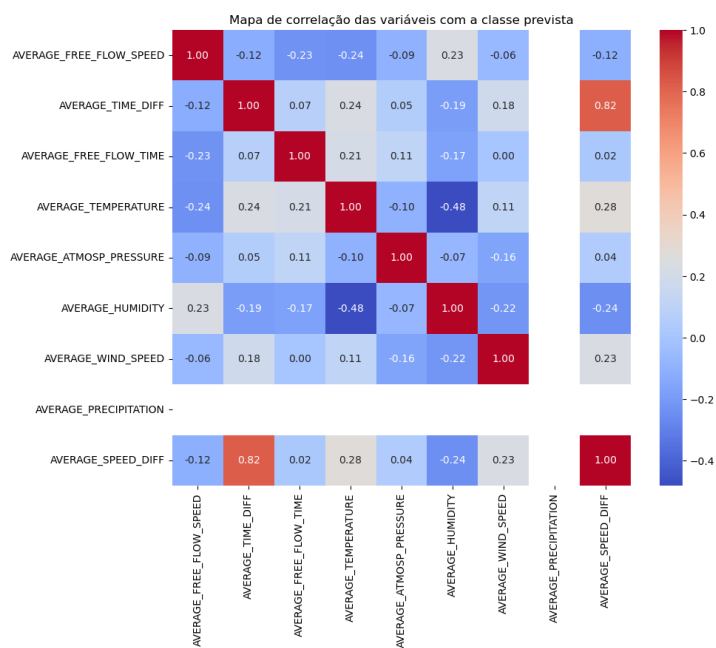


Figura 5.2: Matriz de Correlação das Previsões

As informações retiradas desta pequena análise podem ser usadas para a melhoria dos modelos, e posteriormente muito relevantes para uma melhor gestão da mobilidade urbana e para a mitigação de congestionamentos.

6 Conclusão

O resultado foi positivo: 83.333% como maior pontuação pública, 80.095% como pontuação final pelo modelo escolhido. O grupo terminou assim em 39º lugar, 5 lugares acima antes do fim da competição e com 1.714% de diferença do primeiro lugar.

Crê-se que o resultado poderia ter sido ainda mais positivo se o grupo tivesse dedicado mais tempo a refinar modelos como o Support Vector Machine e as Redes Neurais, modelos poderosos que no entanto registaram grandes descidas. Outra possibilidade seria maior dedicação ao *fine-tuning* do modelo, algo que o grupo tentou implementar mas de forma mais restrita. Com base na análise qualitativa final, poderiam ainda ser feitas melhorias a novos modelos, nomeadamente adicionando pesos.

O grupo conclui o trabalho com sentimento de realização e superação. Ao longo do semestre, os modelos aprendidos em sala de aula foram aplicados ao problema deste trabalho, fornecendo uma aprendizagem rica e empenhada.